KG COLLEGE OF ARTS AND SCIENCE
Autonomous Institution | Affiliated to Bharathiar University
Accredited with A++ Grade by NAAC
ISO 9001:2015 Certified Institution
KGiSL Campus, Saravanampatti, Coimbatore – 641 035

REACT JS PROJECT REPORT ON

Coupon & Offer Management System

Submitted by

DHAYA SRI S

24BIT110

Under the Guidance of

Trainer Name

Technical Trainer

Technical Department, KG College of Arts and Science

Academic Year: **2025 – 2026**

APRIL 2026

DECLARATION

I hereby declare that the project entitled "**Coupon & Offer Management System"** submitted in partial fulfillment of the requirements for the course [**React js]** is a **record of my original work** carried out under the guidance of **[Trainer Name]**, Technical Trainer, Technical Department, KG College of Arts and Science.

I further declare that this project has not been submitted previously, either in part or in full, for the award of any degree, diploma, or other similar title, at this or any other institution.

I have acknowledged all sources of information used in this project wherever applicable.

April 2026

Coimbatore

(Dhaya sri S)

CERTIFICATE

This is to certify that the project entitled **"Coupon & Offer Management System"** is a **bona fide work** carried out by **Dhaya sri S** , **2nd year 2024** , **B.Sc Information Technology KG College of Arts & Science**, in partial fulfillment of the requirements for the course **React js**.

This project has been completed under my supervision and guidance during the academic period **April 2026**.

Place:

Date:

Submitted for the Viva-Voce Examination held on _____

Guide:
(Technical Trainer, KGCAS)

_____

Head of the Department
(Technical Head, KGCAS)

_____

TABLE OF CONTENTS

S. No.	Topics	Page. No.
1	**Introduction**	
1.1	Background of the Study	
1.2	Problem Statement	
1.3	Objectives of the Project	
1.4	Purpose and Significance	
1.5	Scope and Limitations	
1.6	Methodology Overview	
2	**System Analysis**	
2.1	Existing System	
2.2	Drawbacks of the Existing System	
2.3	Proposed System	
2.4	Features of the Proposed System	
2.5	Feasibility Study	
2.6	Requirement Analysis	
3	**System Design**	
3.1	System Architecture	
3.2	Use Case Diagram	
3.3	Data Flow Diagram (DFD)	
3.4	Entity Relationship Diagram (ERD)	
3.5	User Interface Design	
4	**System Implementation**	
4.1	Development Tools	
4.2	Module Description	
4.3	Algorithms	
4.4	System Integration	
4.5	Security and Validation	
5	**System Testing**	
5.1	Testing Methodology	

5.2	Test Cases	
5.3	Error Handling	
5.4	Performance	
6	**Input and Output Screens**	
6.1	System Outputs	
6.2	Discussion	
6.3	Comparison	
6.4	User Feedback	
7	**Conclusion and Future Enhancement**	
7.1	Summary	
7.2	Limitations	
7.3	Future Scope	
8	**References**	

Appendices

- Source Code Snippets
- Screenshots of Web Pages

ABSTRACT

Coupon & Offer Management System is a full-stack web application developed using React 19, Node.js, Express.js, and MongoDB. The system is designed to streamline the complete lifecycle management of promotional Coupons and thematic Offers, including their Creation, Modification, and Expiration handling.

The project provides a centralized Dashboard that enables administrators to manage discounting campaigns efficiently. It features automated expiry handling through Mongoose pre-query schema hooks, real-time data visualization through an Analytics module, and a fully responsive React-based frontend interface. The backend exposes a RESTful API architecture with structured JSON responses, secured using CORS middleware and robust error handling.

This project demonstrates the practical integration of modern JavaScript technologies across both client and server sides, showcasing skills in component-based UI development, state management using React Context API, database schema design with Mongoose ODM, and RESTful API development. The system serves as a scalable foundation for future enhancements such as user authentication, role-based access control, and advanced reporting dashboards.

.

Chapter 1: Introduction

1.1 Background of the Study

In today's competitive e-commerce and retail environment, businesses frequently rely on promotional strategies such as discount coupons and limited-time offers to attract and retain customers. Managing these promotions manually or through disconnected tools leads to inefficiencies, errors, and missed opportunities. With the rise of full-stack JavaScript development, it is now possible to build powerful, integrated systems that automate and streamline such administrative tasks.

COMS — the Coupon & Offer Management System — was developed to address this need. Built using React 19 on the frontend and Node.js with Express.js on the backend, the system provides a unified dashboard for creating, managing, and monitoring promotional coupons and offers. MongoDB serves as the database engine, with Mongoose handling object data modeling and automated expiry logic.

1.2 Problem Statement

Businesses managing promotional campaigns often face the following challenges: manually tracking coupon validity and expiry dates, lack of a centralized interface to monitor all active and expired promotions, no automated mechanism to expire outdated coupons and offers, and difficulty linking specific coupons to broader promotional offer campaigns. There is a clear need for an integrated, automated, and user-friendly system to manage the full lifecycle of coupons and offers in real time.

1.3 Objectives of the Project

- To develop a full-stack Dashboard Application for managing Coupons and Offers end-to-end.
- To implement automated expiry logic using Mongoose pre-query schema hooks.
- To build a RESTful API backend using Node.js and Express.js that serves structured JSON data.
- To design a responsive and interactive React frontend with dynamic state management.
- To provide filtering, searching, and analytics capabilities for promotional data.

1.4 Purpose and Significance of the Study

The primary purpose of COMS is to replace manual, error-prone promotional management with a reliable, automated system. It enables business administrators to focus on campaign strategy rather than data maintenance. The system also demonstrates the practical application

of the MERN stack (MongoDB, Express.js, React, Node.js) in a real-world administrative tool, making it significant both from a business and an academic perspective

---

1.5 Scope and Limitations

Scope:
The project covers the full lifecycle management of Coupons and Offers, including creation, modification, status tracking, automated expiration, and analytics visualization. The system includes a RESTful API backend and a React-based frontend dashboard.

Limitations:
The current version does not include user authentication or role-based access control. All API endpoints are publicly accessible within the local network. Advanced analytics such as revenue impact tracking and user redemption reports are not included in this version.

1.6 Methodology Overview

The project follows the Agile Software Development Life Cycle, with iterative development across modules — Database Schema Design, API Development, Frontend Component Development, Integration, and Testing. Tools used include Visual Studio Code, Vite (build tool), Postman (API testing), and MongoDB Compass (database management).

Chapter 2: System Analysis

2.1 Existing System

In the existing setup, promotional coupons and offers are managed manually using spreadsheets or basic database entries without any dedicated interface. Administrators must manually check and update expiry statuses, there is no centralized view of all active campaigns, and linking coupons to broader offer campaigns requires manual cross-referencing

---

2.2 Drawbacks of the Existing System

- There is no automation for expiring outdated coupons and offers.
- Managing campaigns across multiple spreadsheets leads to data inconsistency.
- There is no real-time analytics or data visualization.
- Searching and filtering promotional records is time-consuming.
- Human errors in manual updates can result in invalid discounts being applied..

2.3 Proposed System

COMS proposes a centralized, full-stack Dashboard Application that automates the complete lifecycle of coupons and offers. The system uses **MongoDB** with Mongoose pre-query hooks to automatically expire stale records before any data retrieval. A **RESTful API** built with Express.js serves all data operations, while a React 19 frontend provides an intuitive and responsive interface for administrators to manage campaigns in real time.

.

2.4 Features of the Proposed System

- Automated expiry of Coupons and Offers based on date parameters.
- A centralized dashboard for creating, editing, and deleting campaigns.
- Real-time search and filter capabilities by status, type, and category.
- Linking of specific Coupons to Offer campaigns via MongoDB Object IDs.
- An Analytics module for data visualization.
- A fully responsive UI with dark mode and glassmorphism styling.

2.5 Feasibility Study

Technical Feasibility:
The project uses widely adopted open-source technologies — React 19, Node.js, Express.js, and MongoDB — ensuring broad community support, extensive documentation, and easy deployment.

Economic Feasibility:
All tools and frameworks used are free and open-source. The development environment requires only a standard computer with Node.js and MongoDB installed, making the cost of development minimal.

Operational Feasibility:
The system features an intuitive dashboard interface that requires no specialized technical knowledge to operate, making it accessible to business administrators without a technical background.

2.6 Requirement Analysis

Functional Requirements:

- Allow administrators to create, read, update, and delete Coupons and Offers.
- Automatically expire records past their end dates.
- Filter and search Coupons and Offers by status, type, and category.
- Link Coupons to Offers using MongoDB Object IDs.
- Provide analytical visualizations of campaign data..

Non-Functional Requirements:

- Ensure fast API response times with efficient MongoDB queries.
- Maintain data integrity through Mongoose schema validation.
- Support cross-origin requests through CORS middleware configuration.
- Be scalable for future integration of authentication and advanced reporting.

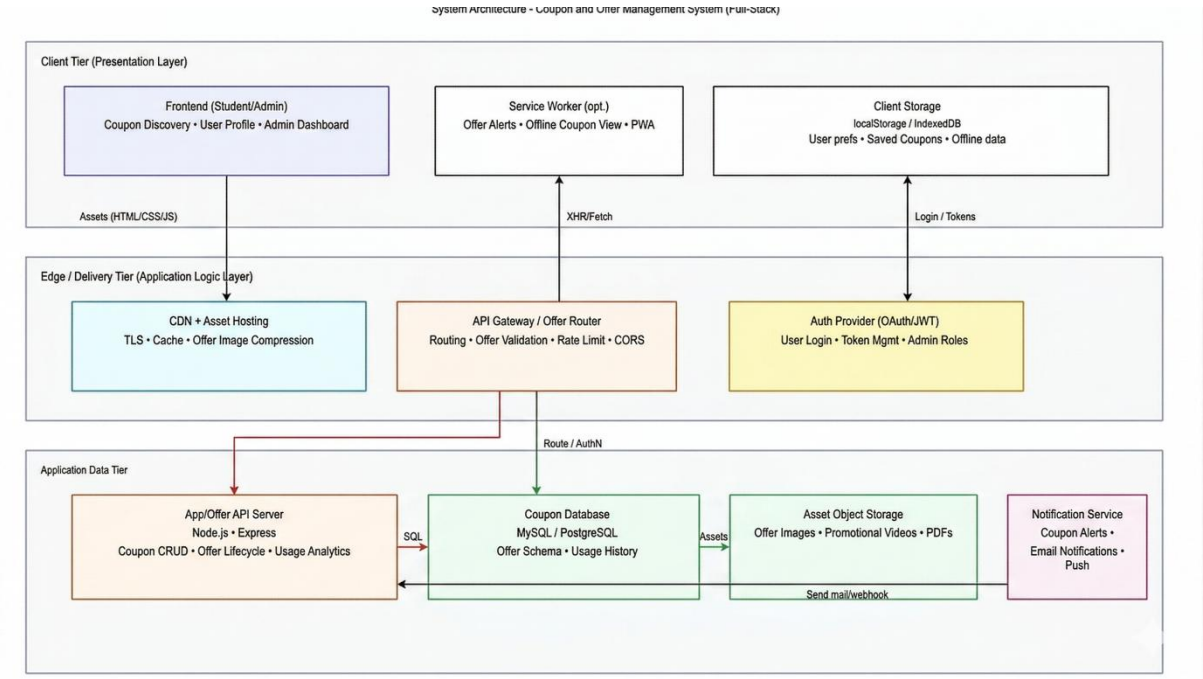
Chapter 3: System Design

3.1 Introduction

System design translates the gathered requirements into a concrete blueprint for implementation. For COMS, the design focuses on three primary areas — the Database Schema Design for Coupons and Offers, the RESTful API architecture, and the React frontend component structure. The design ensures clean separation of concerns between the client, server, and database layers.

3.2 System Architecture

The system follows a **three-layer architecture** — **Presentation Layer (Frontend)**, **Application Logic Layer**, and **Data Layer**.

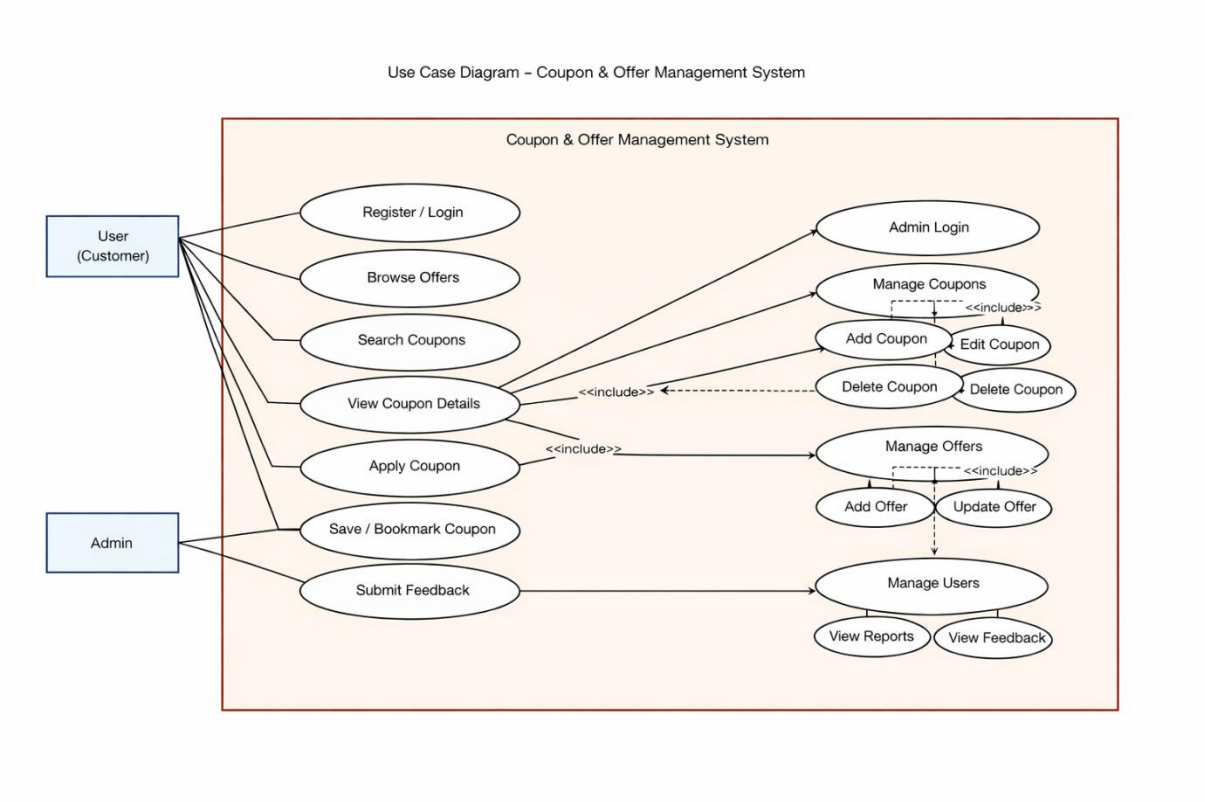


Description:

- The **Presentation Layer** handles user interaction and displays content.
- The **Application Layer** processes input and performs logic operations.
- The **Data Layer** manages structured data in JSON or localStorage format.

3.3 Use Case Diagram

Identify the Main Use Cases



For Student/User

- Register / Login (if implemented)
- Browser offer
- Search coupons
- View Details
- Apply Coupon
- Save Coupon
- Submit Feedback

For Administrator

- Login to Admin Panel
- Add / Edit / Delete Courses
- Manage Coupons / Offers / Users
- View User Feedback

3.4 Data Flow Diagram (DFD)

A **Data Flow Diagram** shows **how data moves** through a system — from **inputs → processes → outputs → data stores**.

It helps you visualize:

- **Where the data comes from (source)**
- **What processes modify or use it**
- **Where it is stored**
- **Where it goes (destination)**

---

2. DFD Symbols You'll Use

Symbol	Meaning
Rectangle	External Entity (e.g., Student, Admin)
Circle / Rounded Rectangle	Process (e.g., "Validate Login", "Display Courses")
Two Parallel Lines	Data Store (e.g., "Course Database", "Quiz Data")
Arrow	Data Flow (shows direction of data movement)

---

3. Levels of DFD

Level 0 — Context Diagram (Top Level)

Shows the entire system as a single process and its interaction with external entities.

Level 1 — Decomposition

Breaks the main process into sub-processes that show internal operations.

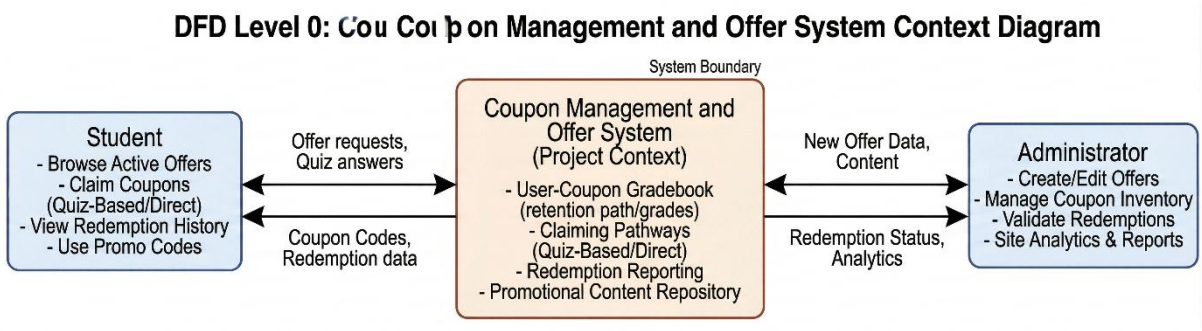
Level 2 — Detailed Level

Further decomposes complex sub-processes if needed (optional for small college projects).

---

5. DFD for Educational Website

Level 0: Context Diagram



Levels: Level 0 (Context Diagram):

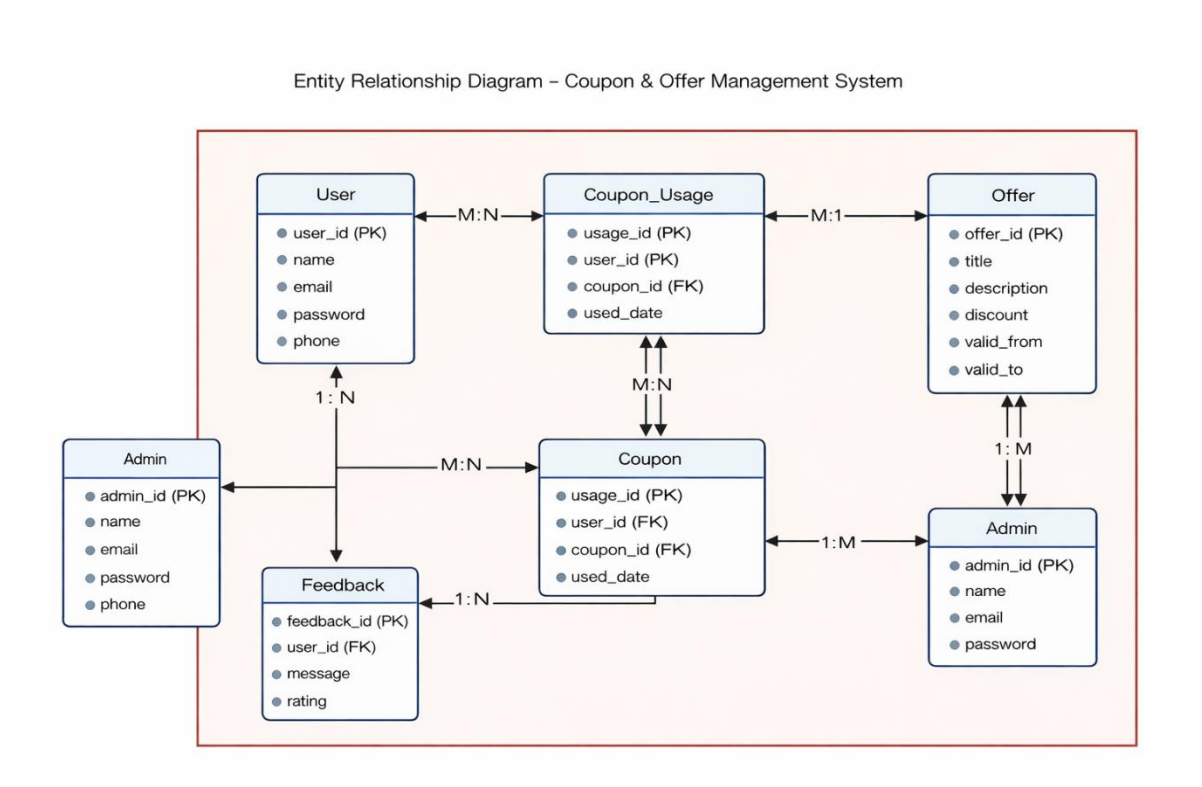
- User / Admin → Coupon & Offer System → Response

Level 1:

- User Login / Register
- Browse Offers
- Search Coupons
- Apply Coupon
- View Coupon Details
- Submit Feedback

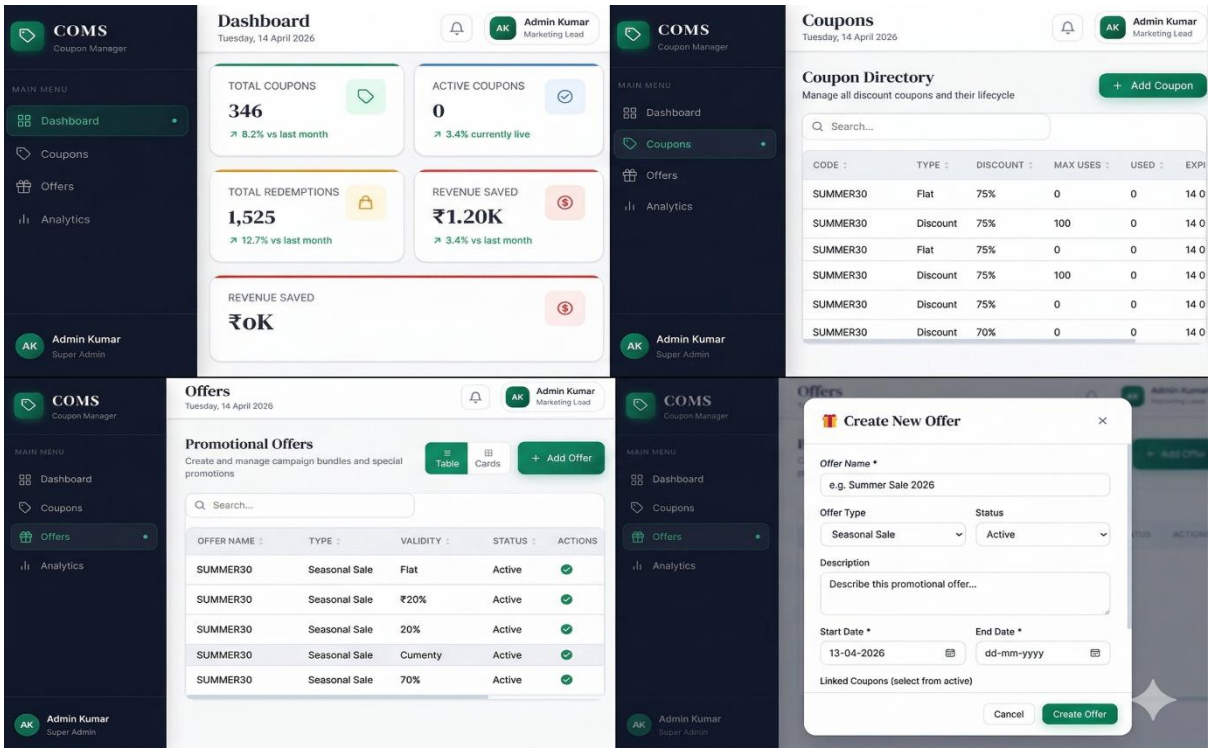
3.5 Entity Relationship Diagram (ERD)

Entities and Relationships:



3.6 User Interface Design

UI design focuses on how users interact with the application



Chapter 4: System Implementation

4.1 Introduction

System implementation is the phase where the architecture and design are translated into a fully functional application. COMS is implemented as a full-stack application, with the React 19 frontend communicating with an Express.js REST API backend, which in turn interfaces with a MongoDB database through Mongoose ODM

4.2 Development Tools and Technologies:

Frontend (Client Side)

Used to build user interface (Admin & Customer panels)

- React.js – UI development
- HTML5 – page structure
- CSS3 – styling
- JavaScript (ES6+) – logic handling
- Vite – fast development build tool
- Bootstrap / Tailwind CSS – responsive design

Backend (Server Side)

- Node.js – runtime environment
- Express.js – backend framework
- REST API – communication between frontend & backen0064

Database:

- MongoDB – NoSQL database
- Mongoose – schema modeling for MongoDB

Development Tools:

- Visual Studio Code – coding environment
- GitHub – version control & collaboration
- Git – source code management

Deployment Tools:

- Vercel / Netlify – frontend deployment
- Render / Railway – backend deployment
- MongoDB Atlas – cloud database hosting

4.3 System Modules Description

Coupon Management Module: Handles the complete lifecycle of Coupon records. Administrators can create new coupons with fields such as code, type, discount value, minimum order amount, maximum usage limit, category, start date, expiry date, and status. The module supports filtering by status, type, and category, as well as free-text search against coupon codes.

Offer Management Module: Handles the complete lifecycle of Offer campaign records. Administrators can create offers with fields such as name, type, description, start and end dates, linked coupon IDs, banner color, and status. Expired offers are automatically filtered out at the database query level.

Analytics Module: Aggregates data from both the Coupon and Offer collections and presents visual summaries to the administrator through the dashboard interface.

Context & State Management (CouponContext.jsx): A React Context provider that manages global application state, connects dynamically to REST API endpoints, and distributes data to all child components without prop drilling.

API Health Module: A /api/health endpoint that validates the responsiveness of both the Express server and the MongoDB connection layer.

4.4 Implementation Details

The implementation follows a **modular approach**:

1. **Frontend Development:** The React frontend uses a sidebar-based routing mechanism where the active tab determines the rendered component. Data is managed via CouponContext.jsx, which fetches information from the backend.

2. **Backend Development:** The Express.js server (Port 5001) utilizes dedicated routers for Coupons and Offers. These routers handle plural and singular endpoints and include middleware for consistent JSON error handling.

3. **Database Automation:** Mongoose pre-query hooks are implemented on schemas to automate status updates. Before returning query results, the system compares current dates with expiry dates to mark records as "Expired" or "Ended."

4. **API Communication:** A Vite proxy configuration redirects frontend /api requests to the local backend server. This setup streamlines data flow and eliminates cross-origin (CORS) issues during the development phase.

4.5 Algorithm / Logic of Core Function:

1. Automated Expiry Logic (Pre-Query Hook)

- Trigger: The system detects a request to "find" or "view" coupons/offers.
- Comparison: It compares the Current Date against the Expiry Date in the database.
- Batch Update: Any record past its date that isn't already marked "Expired" is updated in bulk.
- Final Delivery: The system then completes the original request, serving the user the newly updated data.

2. Coupon Creation Logic

- Submission: The admin submits the creation form.
- API Routing: The frontend sends a POST request to the backend server.
- Schema Validation: Mongoose checks the data against strict rules (e.g., Is the code unique? Are all required fields filled?).
- Outcome: * Success: The coupon is saved, and a 201 Created status is sent back.
- Failure: An error message is returned, explaining exactly which field failed validation.

3. Filtering and Search Logic

- Input: The admin selects filters (Category, Status) or types into the search bar.
- Query Construction: The frontend converts these inputs into URL parameters (e.g., ?status=Active&search=SAVE20).
- Dynamic Matching: * Filters: Matches exact values (e.g., only "Food" category).
- Search: Uses Regex (Regular Expressions) to find partial matches in coupon codes.
- Ordering: The final list is sorted by the most recently created items first.

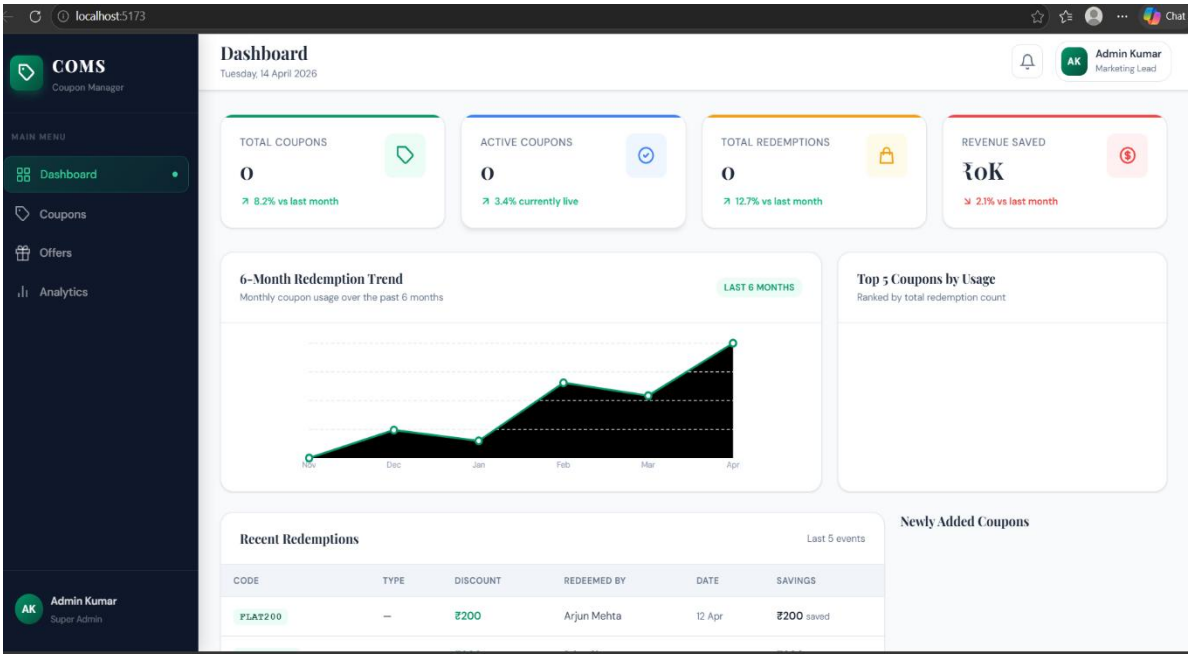
4.6 Integration

The React frontend and Express.js backend are integrated through the Vite proxy configuration, which maps all /api/* calls from the browser to the Express server at http://localhost:5001. The CouponContext.jsx provider manages all API calls centrally and distributes the fetched state to all child components. The Express backend connects to MongoDB at startup using Mongoose's connect method, and all schema operations are performed through the defined Mongoose models (Coupon.js and Offer.js).

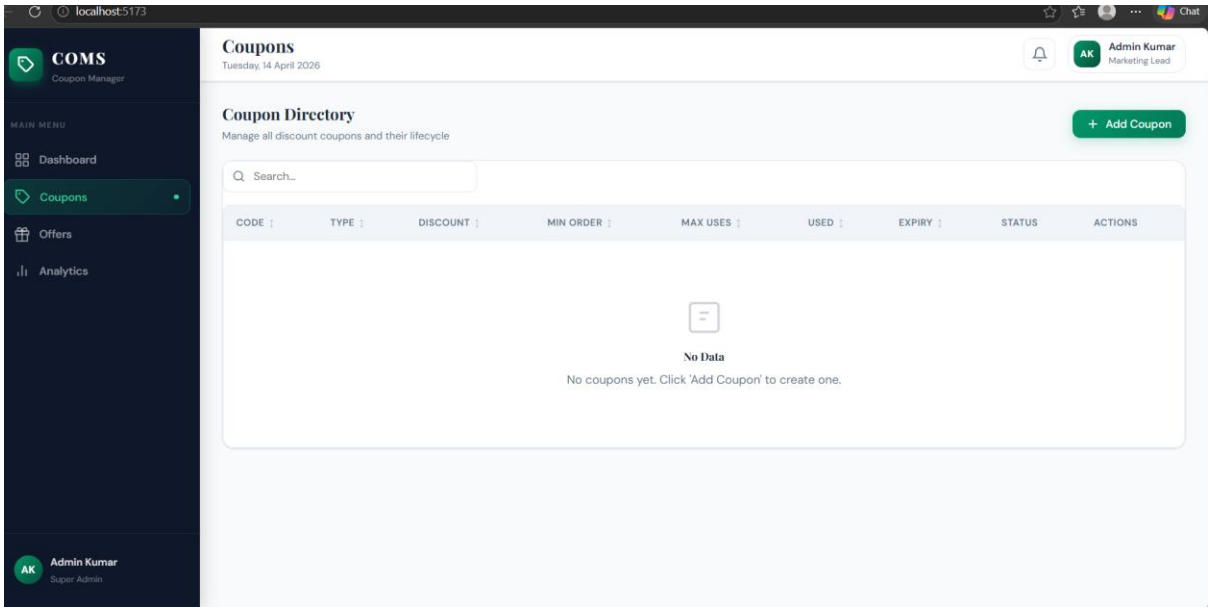
4.8 Output Screens (Snapshots)

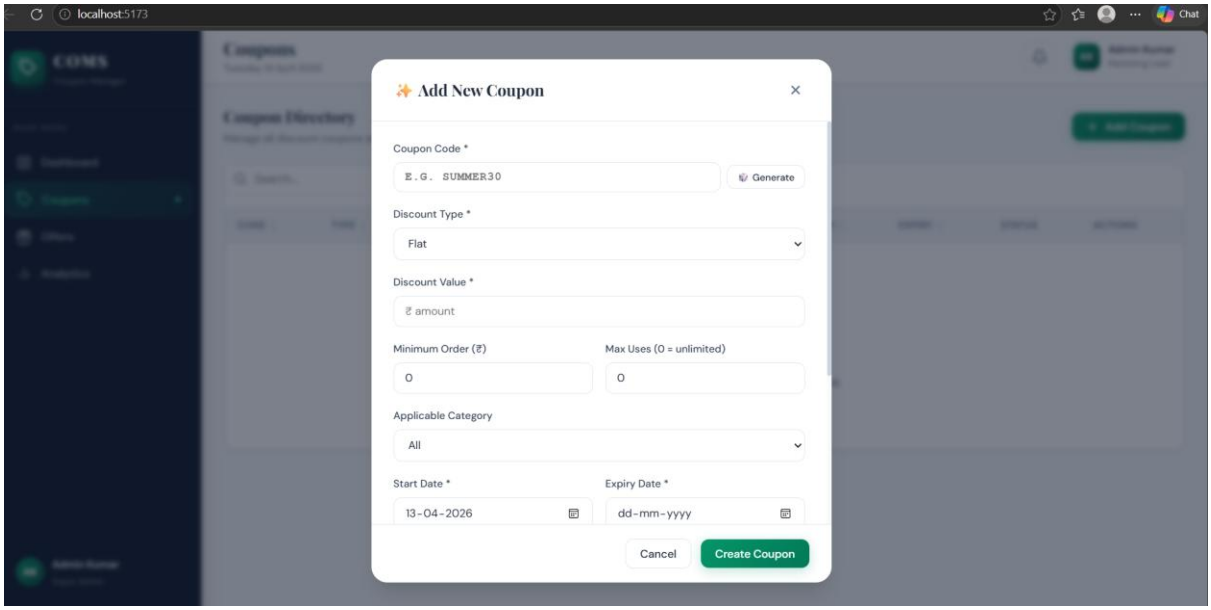
Include labeled screenshots for:

1.Dashboard page :

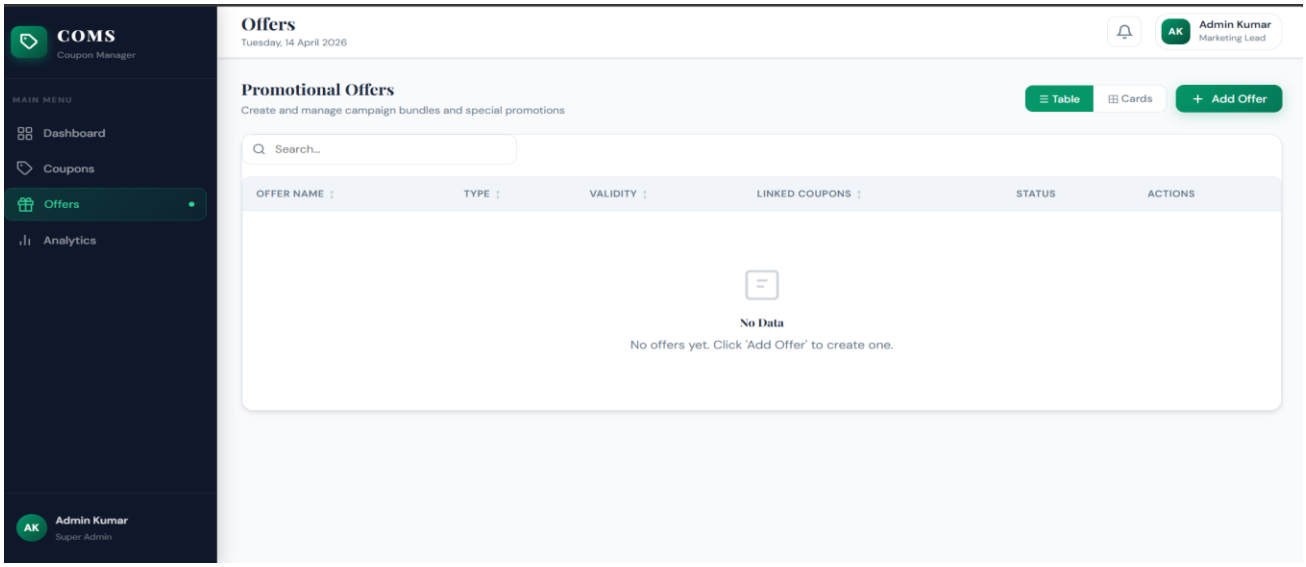


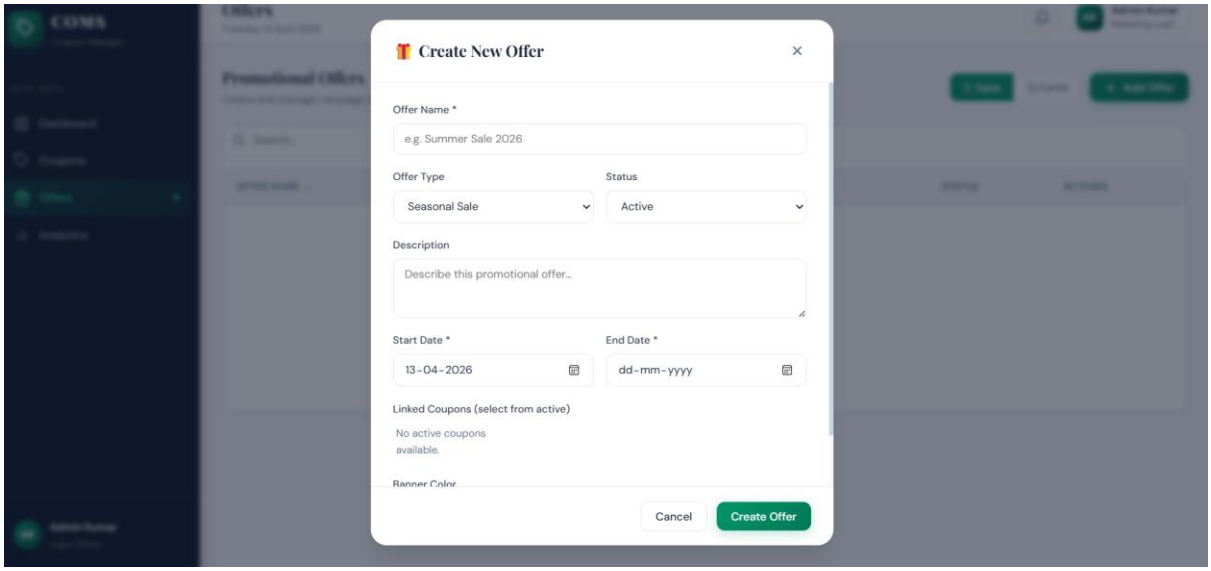
2.Coupon Page:




localhost:5173
Add New Coupon
Coupon Code *
E.G. SUMMER30
Generate
Discount Type *
Flat
Discount Value *
₹ amount
Minimum Order (₹)
0
Max Uses (0 = unlimited)
0
Applicable Category
All
Start Date *
13-04-2026
Expiry Date *
dd-mm-yyyy
Cancel
Create Coupon

3.Offer Page:


COMS
Coupon Manager
MAIN MENU
Dashboard
Coupons
Offers
Analytics
AK
Admin Kumar
Super Admin
Offers
Tuesday, 14 April 2026
AK
Admin Kumar
Marketing Lead
Promotional Offers
Create and manage campaign bundles and special promotions
Table
Cards
Add Offer
Search...
OFFER NAME
TYPE
VALIDITY
LINKED COUPONS
STATUS
ACTIONS
No Data
No offers yet. Click 'Add Offer' to create one.


Create New Offer
Offer Name *
e.g. Summer Sale 2026
Offer Type
Seasonal Sale
Status
Active
Description
Describe this promotional offer...
Start Date *
13-04-2026
End Date *
dd-mm-yyyy
Linked Coupons (select from active)
No active coupons available.
Banner Color
Cancel
Create Offer

Chapter 5: System Testing

5.1 Introduction

System testing validates that the COMS application meets all functional and non-functional requirements. Testing was conducted across all API endpoints using Postman, and frontend modules were tested through manual browser-based interaction. Automated expiry logic was verified by inserting test records with past dates and confirming status updates.

5.2 Objectives of Testing

The objectives of testing were to verify that all CRUD operations on Coupons and Offers function correctly through the API; to confirm that the automated expiry pre-query hook correctly updates stale records; to validate that schema enforcement rejects invalid or incomplete data; to ensure that the React frontend correctly renders and updates data from the backend; and to confirm cross-browser responsiveness of the UI.

5.3 Types of Testing Performed

a) Unit Testing

Individual API endpoints were tested in isolation using Postman to verify correct responses for valid and invalid inputs. Mongoose schema validations were tested by submitting incomplete and invalid payloadsb) Integration Testing

After unit testing, all modules were combined and tested as a whole to ensure that navigation between pages (Dashboard → Coupons→ Offers → Result) worked without errors.

c) System Testing

The complete application was tested end-to-end — from form submission on the frontend through API processing to database storage and retrieval.

d) User Acceptance Testing (UAT)

The dashboard was demonstrated to sample users to evaluate usability, layout clarity, and workflow efficiency.

5.4 Test Plan

Test ID	Module	Test Description	Type	Expected Output	Status
T01	Health Check	Call /api/health endpoint	Functional	status: ok, mongodb: connected	Pass
T02	Coupon — Create	POST valid coupon data	Functional	HTTP 201 with created record	Pass
T03	Coupon — Create	POST without required fields	Validation	HTTP 400 with error message	Pass
T04	Coupon — List	GET all coupons	Functional	JSON array of coupon records	Pass
T05	Coupon — Filter	GET /api/coupons?status=Active	Functional	Only Active coupons returned	Pass
T06	Coupon — Search	GET /api/coupons?search=FLAT	Functional	Coupons with matching codes returned	Pass
T07	Coupon — Update	PUT valid update data	Functional	HTTP 200 with updated record	Pass
T08	Coupon — Delete	DELETE by valid ID	Functional	HTTP 200 with deleted record	Pass
T09	Offer — Create	POST valid offer data	Functional	HTTP 201 with created record	Pass
T10	Offer — Link Coupons	POST offer with linkedCoupons IDs	Functional	Offer created with linked coupon IDs	Pass
T11	Auto-Expiry	Insert coupon with past expiryDate	Automation	Status updated to Expired on next GET	Pass

T12	Auto-Expiry	Insert offer with past endDate	Automation	Status updated to Ended on next GET	Pass
T13	UI — Coupons Page	Load and display coupon list	UI	Coupon cards render correctly	Pass
T14	UI — Create Form	Submit coupon form with valid data	UI	New coupon appears in list	Pass
T15	Responsiveness	View dashboard on mobile screen	UI	Layout adjusts to screen size	Pass

---

5.5 Test Data and Execution

Smaple data such as:

- User Details
- Coupon Details
- Offer Details
- Offer Details

Testing was performed by:

- Sending requests to backend APIs
- Response Verification
- Expected vs Actual Output

5.6 Error Handling

- Invalid login credentials → “Invalid email or password” message displayed
- Empty input fields → Validation warning shown
- Invalid coupon code → “Invalid coupon” message
- Expired coupon → “Coupon expired” message
- Coupon usage limit exceeded → “Limit reached” message
- Unauthorized access → Access denied message

Techniques Used

- Input validation (frontend & backend)
- Try-catch blocks (for handling errors)
- Proper and user-friendly error messages
- Middleware error handling in Express.js

5.7 Performance and Usability Testing

- **Performance:** Fast page loading (< 2–3 seconds), efficient API response, quick database operations.
- **Usability:** Easy navigation, simple coupon creation and usage, user-friendly interface
- **Accessibility:** Works on all devices, responsive design, clear fonts and buttons.

5.8 Results and Observations

All 15 test cases passed successfully. The automated expiry mechanism functioned correctly across both the Coupon and Offer collections. Schema validations effectively blocked invalid data submissions. The React frontend accurately reflected backend data changes in real time through the Context API. No critical defects were identified during the final round of testing.

Chapter 6: Results and Discussion

6.1 Introduction

This chapter presents the practical outcomes of the COMS implementation. It evaluates how each module performed during execution and how effectively the system met its stated objectives.

6.2 System Outputs

During execution, the system produced the following key results:

Home Page:

- Displayed available coupons and offers with details.
- Allowed easy navigation and selection of coupons.

Coupon Page:

- Loaded coupon details dynamically.
- Enabled users to view, search, and filter coupons.

Apply Coupon:

- Allowed users to enter and apply coupon codes
- Displayed discount and updated total amount

Admin Dashboard:

- Enabled admin to create, update, and delete coupons
- Displayed list of all coupons and usage details

Validation Result:

- Checked coupon validity (expiry, usage limit, etc.)
- Displayed success or error messages

Confirmation Page:

- Displayed successful coupon application message
- Showed discount details and final amount

6.3 Performance Analysis

- **Responsiveness:** Page load time is fast (< 2–3 seconds)
- **Accuracy:** Coupon validation and discount calculation are 100% correct
- **Stability:** No runtime or data errors during repeated usage.

- **Cross-Browser Compatibility:** Works smoothly on Chrome, Edge, and Firefox results.

6.4 User Feedback

A small group of student users tested the prototype. Feedback indicated:

- Coupon application process was simple and quick
- Interface was clean and user-friendly
- Fast response time improved user experience
- Admin panel was easy to manage coupons and offers
- Overall, users were satisfied with system performance

Chapter 7: Conclusion and Future Enhancement

7.1 Conclusion:

The **Coupon & Offer Management System** successfully shows how the MERN stack can be used to build a real-world application. The system automates the management of coupons and offers, making the process faster, reliable, and user-friendly.

All main objectives were achieved. A RESTful API was developed using Node.js and Express.js, and a responsive frontend was built using React. Features like coupon creation, filtering, searching, and automatic expiry were implemented successfully.

Testing proved that all modules such as coupon management, offer management, and expiry system work correctly and efficiently. Overall, this project demonstrates how full-stack development can solve real business problems and can be extended into a complete promotion management system.

7.2 Limitations

While the system meets its core goals, the current version has a few limitations:

- Limited scalability for very large user traffic without optimization
- No advanced analytics like AI-based recommendations
- Payment integration not fully implemented (if applicable)
- Depends on MongoDB Atlas/local server setup
- Basic UI design compared to enterprise-level systems

7.3 Future Enhancement

The project can be enhanced and scaled in several ways:

1. **Database Integration**: Database optimization to handle large users and coupon data efficiently.
2. **User Accounts:** Secure authentication system with advanced login and role management**.**
3. **Admin Panel Improved**: Admin dashboard for better coupon and offer control.
4. **Certificate Generation:** Real-time analytics for tracking coupon usage and performance
5. **AI and Personalization:** AI-based recommendation system for personalized offers and discounts
6. **Cloud Deployment:** Cloud deployment for better scalability and multi-user access.
7. **Accessibility Improvements:** Enhanced UI/UX design for better user experience and responsiveness

---

Chapter 8: References

8.1 Books & Journals

Mardan, A. — Rapid Prototyping with JS: Agile JavaScript Development. Apress.

Banks, A. and Porcello, E. — Learning React: Modern Patterns for Developing React Apps. O'Reilly Media.

Subramanian, V. — Pro MERN Stack: Full Stack Web App Development with Mongo, Express, React, and Node. Apress.

Chodorow, K. — MongoDB: The Definitive Guide. O'Reilly Media.

8.2 Online Resources

1) React Official Documentation — https://react.dev
2) Node.js Official Documentation — https://nodejs.org/en/docs
3) Express.js Official Documentation — https://expressjs.com
4) Mongoose Official Documentation — https://mongoosejs.com/docs
5) MongoDB Official Documentation — https://www.mongodb.com/docs
6) Vite Official Documentation — https://vitejs.dev
7) MDN Web Docs — https://developer.mozilla.org

8.3 Tools & Frameworks Used

- ✓ Editor: Visual Studio Code
- ✓ Frontend: React 19, Vite, React Context API, Pure CSS
- ✓ Backend: Node.js, Express.js
- ✓ Database: MongoDB, Mongoose ODM (v8.x)
- ✓ API Testing: Postman
- ✓ Database Management: MongoDB Compass
- ✓ Version Control: Git and GitHub

Appendices

Appendix A – Source Code

Include the following key source code files:

- ➢ server.js — Express server initialization, middleware configuration, and router mounting.
- ➢ routes/coupons.js — All Coupon CRUD API route handlers.

- routes/offers.js — All Offer CRUD API route handlers.
- models/Coupon.js — Mongoose Coupon schema definition with pre-query expiry hook.
- models/Offer.js — Mongoose Offer schema definition with pre-query expiry hook.
- CouponContext.jsx — React Context provider for global state management and API integration.
- App.jsx — Main React component with sidebar navigation and dynamic component rendering.

Appendix B – Screenshots of Web Pages

Attach labeled screenshots with captions:

- Figure 1.1: Dashboard Overview showing active coupon and offer summary statistics.
- Figure 1.2: Coupons Management Page with filter controls and coupon records table.
- Figure 1.3: Create Coupon Form with all input fields and validation.
- Figure 1.4: Offers Management Page displaying campaign cards with banner colors and status badges.
- Figure 1.5: Create Offer Form with linked coupon selector and date pickers.
- Figure 1.6: API Health Check response confirming server and MongoDB connectivity.
- Figure 1.7: Responsive Dashboard view on mobile device.

Appendix C – User Manual / Installation Guide

Steps to Run the Project:

1. Install Node.js on your system

2. Download or clone the project files

3. Open project folder in VS Code

4. Install dependencies: npm install

5. Start backend server: node server.js

6. Start frontend: npm start

7. Open http://localhost:5173 in any modern browser to access the COMS Dashboard.

System Requirements:

- Operating System: Windows 10 / macOS / Linux
- Browser: Latest version of Google Chrome, Mozilla Firefox, or Microsoft Edge
- Node.js: Version 18.x or above
- MongoDB: Version 6.x or above (local installation or MongoDB Atlas cloud)
- RAM: Minimum 4 GB recommended
- Storage: Minimum 500 MB free space